AI Coding 与 tmux 最佳实践指南

AI Agent（Claude Code、Cursor、Aider 等）通常具备：长时运行、需交互确认、产生大量日志的特点。 `tmux` 的会话持久化与窗口管理，恰好能解决这些痛点。
本指南聚焦 Claude Code，但思路同样适用于其他 CLI / Agent 场景。

一、为什么用 tmux 跑 AI Agent?

需求场景	传统方案（nohup / systemd）	tmux 方案
断网 / 关终端	进程可能随 Shell 退出而终止	进程持续运行，完全不受影响
Agent 请求确认	无法交互，导致 Agent 卡死	随时重连，即时输入 `y/n`
监控进度	需 `tail -f` 查看日志，无法看到全貌	分屏实时查看输出与代码变更
环境复原	需手动重启	恢复会话，环境状态完整保留

二、Claude Code 权限模式速览（含 `--dangerously-skip-permissions`）

Claude Code 通过 **权限模式** 控制是否在执行命令前弹窗确认。核心模式包括：

模式	行为概述	典型用途
`default`	标准行为：首次使用每个工具时询问	入门、敏感项目
`acceptEdits`	自动允许文件编辑，仍询问 Bash	已熟悉的代码重构
`plan`	只读分析，不修改文件也不执行命令	探索代码库、 制定重构方案
`auto`	后台安全分类器审核后自动执行，减少弹窗	长任务、 降低交互频率

模式	行为概述	典型用途
`bypassPermissions`	跳过权限提示与安全检查，仅对写 `.git/.vscode/.idea` 等目录保留确认	**仅限隔离环境（容器 / VM）**

`--dangerously-skip-permissions` 是什么？

- **功能**：等价于 `--permission-mode bypassPermissions` ，启用“跳过所有权限检查”的自治模式。
- **典型命令**【turn2fetch0】：

```
# 最常见用法
claude --dangerously-skip-permissions "Fix all lint errors"

# 等价写法
claude --permission-mode bypassPermissions "Fix all lint errors"
```

- **关键行为**：
  - 大部分工具调用不再弹窗，直接执行。
  - 仍会对写 `.git` 、 `.vscode` 、 `.idea` 等目录弹出确认，避免破坏仓库与 IDE 配置。
  - **不提供针对提示注入或误操作的安全防护**；安全依赖外部环境（容器、沙箱等）。

何时应该 / 不应该使用？

适合使用（具备隔离条件时）【turn2fetch0】：

- 在 **Docker 容器 / devcontainer / VM** 中执行自动化任务：
  - 批量修复 lint 错误
  - 标准脚手架生成
  - 大规模自动化重构
- CI/CD 流水线中，在可控环境里跑一次性任务。

不建议使用（高风险场景）【turn2fetch0】：

- 在 **本地主系统**、生产服务器、或包含敏感数据的环境直接运行。
- 任务描述非常宽泛，例如“改进代码”“做任何必要修改”。
- 仓库或环境中存在 `.env` 、密钥、生产配置等敏感信息。

安全使用建议

1. **始终在隔离环境中运行**【turn2fetch0】
   如 Docker 容器 + 无网络：

   ```
   docker run -it --rm \
     -v $(pwd):/workspace \
     -w /workspace \
     --network none \
     claude-code:latest \
     --dangerously-skip-permissions "Fix all ESLint errors in src/"
   ```

   `--network none` 可防止数据外泄【turn2fetch0】。
2. **使用版本控制作为“安全网”**【turn2fetch0】

   ```
   # 启动前先打一个检查点
   git add -A && git commit -m "checkpoint: before autonomous mode"

   # 若 Agent 跑偏，可随时回滚
   git reset --hard HEAD
   ```

3. **尽量用更精细的权限配置替代 `--dangerously-skip-permissions`** 【turn2fetch1】
   在 `.claude/settings.json` 或 `~/.claude/settings.json` 中配置：

   ```
   {
     "permissions": {
       "defaultMode": "acceptEdits",
       "allow": ["Bash(npm run *)", "Bash(git diff *)", "Read", "Edit(/src/**)"],
       "deny": ["Bash(rm -rf *)", "Bash(sudo *)", "Read(./.env*)", "Edit(./.env*)"]
     }
   }
   ```

   即便 `acceptEdits` 模式下， `deny` 规则仍然生效，可显著降低风险【turn2fetch1】。
4. **禁止在 root / sudo 环境下使用**
   Claude Code 会在检测到 root / sudo 时拒绝运行 `--dangerously-skip-permissions` ，这是官方安全限制【turn2fetch2】。

三、tmux 基础工作流：让 Agent 在后台“永生”

1. 启动命名会话

```
# 创建并进入会话
tmux new -s claude-dev

# 在会话中启动 Claude Code
claude --permission-mode acceptEdits
```

需要长时间无人值守时，可在隔离环境中使用：

```
claude --dangerously-skip-permissions "Migrate database schema"
```

2. 分离会话（让 Agent 后台运行）

- **快捷键**： `Ctrl+B -> d` （小写）
  这是最常用操作，务必记住是小写 `d` 。

3. 随时“重逢”查看进度

```
# 列出所有后台会话
tmux ls

# 重新连接到指定会话
tmux attach -t claude-dev
```

如果 Agent 在 `bypassPermissions` 模式下仍然偶尔弹出确认，可以通过 `tmux attach` 及时响应，然后再分离继续跑。

四、tmux 高阶布局：打造 Agent 指挥中心

一键启动脚本 `start_agent.sh`

```bash
#!/bin/bash
SESSION="agent-work"

# 如果会话不存在则创建
tmux has-session -t $SESSION 2>/dev/null

if [ $? != 0 ]; then
  # 1. 创建会话并运行 Claude Code（示例：acceptEdits 模式）
  tmux new-session -s $SESSION -d 'claude --permission-mode acceptEdits'

  # 2. 垂直分割窗格（左右）
  tmux split-window -h -t $SESSION

  # 3. 在右侧窗格运行监控命令
  tmux send-keys -t $SESSION 'htop' C-m   # 或：watch -n 1 git status

  # 4. 光标切回左侧
  tmux select-pane -t $SESSION -L
fi

# 连接进入
tmux attach -t $SESSION
```

布局效果：

```
+----------------------+----------------------+
|                      |                      |
| Claude Code 运行中   |   htop / git status  |
| (主交互区)           |   (资源/变更监控)    |
|                      |                      |
+----------------------+----------------------+
```

五、保活策略：合盖不断电

关键区分：
`tmux` 只保证会话不因 SSH 断开或终端关闭而退出；**不能阻止操作系统睡眠**。

若笔记本进入“睡眠 / 休眠”，CPU 停止，tmux 中的 Agent 自然也会暂停。要让合盖后 Agent 继续运行，需配置操作系统：

1. Windows

- 控制面板 → 硬件和声音 → 电源选项
  “选择关闭盖子的功能” → 接通电源时选择“不采取任何操作”。

2. Linux（systemd）

编辑 `/etc/systemd/logind.conf`：

```
HandleLidSwitch=ignore
HandleLidSwitchDocked=ignore
```

然后：

```
sudo systemctl restart systemd-logind
```

3. macOS

- 官方支持：**合盖模式（Clamshell）**
  需同时满足：电源 + 外接显示器 + 外接键鼠，方可保持唤醒【turn2fetch0】。
- 非官方：使用第三方工具（如 Amphetamine）阻止休眠，但需注意散热与安全风险。

六、tmux 快捷键精要（大小写敏感）

默认前缀键：`Ctrl+B`。

关键区分： `d` 与 `D`

操作	快捷键	功能
分离当前会话	`Ctrl+B -> d`	**最常用：让会话后台运行**
管理其他客户端	`Ctrl+B -> D` （Shift+d）	列出并踢出其他登录同一会话的客户端

高频操作速查

功能	快捷键	记忆提示
垂直分屏（左右）	`Ctrl+B -> %`	竖线	需 Shift，这里用 `%`
水平分屏（上下）	`Ctrl+B -> "`	双引号需 Shift
切换窗格	`Ctrl+B` -> 方向键	直觉操作
查看所有会话	`Ctrl+B -> s`	Sessions
关闭当前窗格	`Ctrl+B -> x`	Kill

七、实战组合示例

示例 1：本地开发 + 适度自治

```
# 启动 tmux 会话
tmux new -s dev

# 使用 acceptEdits + deny 规则，避免误删敏感文件
claude --permission-mode acceptEdits
```

在 `.claude/settings.json` 中配置 `deny` 规则，避免 `rm` 、 `sudo` 、访问 `.env` 等。

示例 2：容器中完全自治（CI 或一次性任务）

```
# 启动 tmux（便于中途 attach）
tmux new -s ci-job

# 在容器中运行无网络 + 跳过权限
docker run -it --rm \
  -v $(pwd):/workspace \
  -w /workspace \
  --network none \
  claude-code:latest \
  --dangerously-skip-permissions \
  "Fix all ESLint errors and generate a report"
```

示例 3：Plan 模式 + 沙箱预演

```
tmux new -s refactor

# 先用 Plan 模式看方案
claude --permission-mode plan "Migrate auth from JWT to Paseto"

# 确认无误后，在隔离容器中用 bypassPermissions 执行
docker run ... claude --dangerously-skip-permissions "Apply migration plan"
```

八、总结

1. **交互优先**：AI Agent 需要交互确认， `tmux` 完胜 `nohup` 。
2. **权限模式分级**：优先使用 `acceptEdits` / `plan` / `auto` + `deny` 规则；
   `--dangerously-skip-permissions` 仅限隔离环境，切勿在主环境直接使用。
3. **系统与软件分工**： `tmux` 保会话，操作系统设置保供电。
   合盖前务必确认系统已设置为“不睡眠”或使用外接显示器 + 电源的合盖模式。